**Department of Computer Science and Engineering**

CSA06 – Design and Analysis of Algorithms

| Field | Details |
|---|---|
| Assessment | CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform |
| Student Reg. No. | 192425121 |
| Student Name | DRAVID V D |
| Assumption | Servers execute $10^8$ basic operations per second; all logarithms are base 2; n = 1,000,000 is the target catalogue size. |

## (a) Understand the Problem

Input size, dominating basic operation and expected growth of each module:

| Module | Input size | Basic operation (where) | Expected growth / justification |
|---|---|---|---|
| M1 | n = number of products | Title & seller comparison in the IF of the inner loop (executed for every pair i<j) | Quadratic. Every pair is compared, so the count is about $n^2/2$. Growing n 10× grows work 100×. |
| M2 | n = request-window limit | Audit(Request[j]) inside the inner FOR | Linear. The inner loop runs i times (1, 2, 4, …), a geometric series dominated by its last term (~n), not n times per outer pass. |
| M3 | n = entries in the catalogue / index | Key comparison with an entry | S1: linear (up to n). S2: logarithmic (about log n), since half the index is discarded after each comparison. |
| M4 | n = number of customer–product records | The recursive splitting plus linear merge (n units of work per call); recursion tree decides growth | Current (4 subproblems of n/2): quadratic, leaf count is $n^2$. Redesign (2 subproblems): n log n. |
| M5 | n = number of sales records | Record comparison in the sort done at every level (n log n per call) | n $\log^2$ n: log n levels, each costing a sum of sort work ≈ n log n or less. Slightly worse than n log n. |
| Report generator | n = records processed | Arithmetic/record operations; term $3n^2$ dominates | Quadratic. 10n log n and 50 are lower-order terms. |

## (b) Analyse the Iterative Modules

### Module M1 – Duplicate-Listing Detector

**Basic operation:** the comparison Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller.

For a fixed i, the inner loop runs from j = i+1 to n−1, i.e. (n−1) − (i+1) + 1 = n − 1 − i times. The count does not depend on the data (no early exit), so best = average = worst.

$$C(n) = \Sigma\ (i = 0 \text{ to } n-2)\ (n - 1 - i)$$
$$= (n-1) + (n-2) + \ldots + 2 + 1$$
$$= (n-1)\cdot n / 2$$
$$C(n) = n(n-1)/2 = (n^2 - n)/2$$

**Time complexity:** $\Theta(n^2)$ (since $n^2/4 \le (n^2-n)/2 \le n^2/2$ for $n \ge 2$). **Space complexity:** $\Theta(1)$ extra space (only i, j and a counter; the input array itself is not counted).

Check: n = 100,000 gives 4,999,950,000 comparisons ≈ 50 s; n = 1,000,000 gives 499,999,500,000 comparisons ≈ 5,000 s.

### Module M2 – API Rate Limiter

**Basic operation:** Audit(Request[j]).

The outer variable takes the values $i = 2^0, 2^1, 2^2, \ldots$ while $i < n$. Let m be the number of outer iterations; then $2^{(m-1)} < n \leq 2^m$, so $m = \lceil \log_2 n \rceil$. In the iteration where $i = 2^k$, the inner loop runs $i = 2^k$ times ($j = 0 \ldots i-1$).

$$A(n) = \Sigma_{k=0 \text{ to } m-1} 2^k$$
$$= 2^m - 1 \quad \text{(geometric series)}$$
$$A(n) = 2^{\lceil \log_2 n \rceil} - 1$$

For n a power of 2 this is exactly $n - 1$. In general $n - 1 \leq A(n) \leq 2n - 3$ ($n \geq 2$), so $A(n) = \Theta(n)$.

**Is the team member's claim Θ(n log n) correct? No.** The claim is only an upper bound argument: it treats the inner loop as running n times in each of the log n passes. But the inner loop runs i times, not n times, and i doubles each pass. The sum is $1 + 2 + 4 + \ldots + 2^{(m-1)}$, a geometric series dominated by its last term, so it is linear. O(n log n) is technically true (it is a valid but loose upper bound), but it is not tight and Θ(n log n) is false. **Tight bound: Θ(n).**

**Verification for n = 1,024:** i takes values 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 (10 iterations, as 1024 is not < 1024). Total Audit calls = $1+2+4+\ldots+512 = 2^{10} - 1 =$ **1,023 = n − 1**. The claimed n log n would be 1,024 × 10 = 10,240, about 10× too large. This is confirmed by the program in the Appendix.

**Space complexity:** Θ(1).

### (c) Solve a Recurrence by Substitution – Search Version S2

Recurrence: $T(n) = T(n/2) + 1$, $T(1) = 1$ (take $n = 2^k$).

$$T(n) = T(n/2) + 1$$
$$= [T(n/4) + 1] + 1 \quad = T(n/4) + 2$$
$$= [T(n/8) + 1] + 2 \quad = T(n/8) + 3$$
$$\ldots$$
$$= T(n/2^k) + k \quad \text{(general step after k substitutions)}$$

The recursion stops when $n/2^k = 1$, i.e. $k = \log_2 n$. Then:

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n$$

**Exact closed form: $T(n) = \log_2 n + 1$** (for general n: $\lfloor \log_2 n \rfloor + 1$). Proof by induction: $T(1) = 1 + 0 = 1$ ✓; $T(n) = T(n/2) + 1 = (\log_2(n/2) + 1) + 1 = \log_2 n + 1$ ✓.

| Implementation | Time complexity | Space complexity |
|---|---|---|
| Iterative binary search | Θ(log n) | Θ(1) – only low, high, mid |
| Recursive binary search | Θ(log n) | Θ(log n) – call stack depth is about $\log_2 n + 1$ frames (Θ(1) only if the compiler eliminates tail calls) |

For n = 1,000,000, S2 needs at most 20 comparisons.

### (d) Apply the Master Theorem

Master Theorem: for $T(n) = aT(n/b) + f(n)$ compare f(n) with

n^(log_b a). Case 1: f(n) = O(n^(log_b a − ε)) ⇒ Θ(n^(log_b a)). Case 2: f(n) = Θ(n^(log_b a)) ⇒ Θ(n^(log_b a) log n). Case 3: f(n) = Ω(n^(log_b a + ε)) and regularity condition ⇒ Θ(f(n)).

### M4 – Recommendation Engine

| | **Current design: T(n) = 4T(n/2) + n** | **Redesign: T(n) = 2T(n/2) + n** |
|---|---|---|
| a, b | a = 4, b = 2 | a = 2, b = 2 |
| f(n) | n | n |
| n^(log_b a) | $n^{(\log_2 4)} = n^2$ | $n^{(\log_2 2)} = n$ |
| Comparison | f(n) = n = O(n^(2−ε)) with ε = 1 (polynomially smaller) | f(n) = n = Θ(n) (same order) |
| Case | Case 1 | Case 2 |
| Result | T(n) = Θ(n²) | T(n) = Θ(n log n) |
| Exact (T(1)=1) | 2n² − n | $n \log_2 n + n$ |

**Effect of doubling the data.** Current design: T(2n)/T(n) → 2²= 4, so the cost quadruples. Redesign: T(2n)/T(n) = 2n log(2n)/(n log n) = 2(1 + 1/log n), slightly more than 2×, so the cost roughly doubles. Removing replication therefore changes the growth from quadratic to near-linear, and each additional doubling of data gets progressively cheaper relative to the old design.

### M5 – Sales Analytics Engine: T(n) = 2T(n/2) + n log n

Here a = 2, b = 2, f(n) = n log n and n^(log_b a) = n.

- **Case 1:** needs f(n) = O(n^(1−ε)). n log n grows faster than n, so it fails.
- **Case 2:** needs f(n) = Θ(n). n log n is not Θ(n), so it fails.
- **Case 3:** needs f(n) = Ω(n^(1+ε)) for some ε > 0. But n log n = o(n^(1+ε)) for every ε > 0 (log n grows slower than any n^ε), so it fails.

**None of the three standard cases applies** (f(n) is larger than n^(log_b a) only by a logarithmic factor, a gap the basic theorem does not cover).

**Extended Master Theorem:** if f(n) = Θ(n^(log_b a) · log^k n) with k ≥ 0, then T(n) = Θ(n^(log_b a) · log^(k+1) n). Here k = 1, so T(n) = Θ(n log² n).

**Recursion-tree verification:** at level i (i = 0 … log n − 1) there are 2^i nodes, each costing (n/2^i) log(n/2^i), so the level costs n(log n − i).

$$\Sigma_{i=0}^{\log n - 1} n(\log n - i) = n(\log n + (\log n - 1) + \ldots + 1)$$
$$= n \cdot \log n (\log n + 1) / 2$$

Leaves: n · T(1) = Θ(n)

T(n) = n log n (log n + 1)/2 + Θ(n) = Θ(n log² n)

### (e) Best, Average and Worst Cases

| **Case** | **Input condition** | **S1 (sequential) comparisons** | **S2 (binary) comparisons** |
|---|---|---|---|
| Best | S1: key is the first element. S2: key is the middle element | 1 → Θ(1) | 1 → Θ(1) |
| Average | Key present, equally likely at any position | (n+1)/2 → Θ(n) | $\approx \log_2 n - 1$ → Θ(log n) |
| Worst | S1: key is last or absent. S2: | n → Θ(n) | $\lfloor \log_2 n \rfloor + 1$ → Θ(log |

| | | | |
|---|---|---|---|
| | key is found at the last level or is absent | | n) |

**S1 – average cases with probability p of the product being present**

Assume that if the product is present it is equally likely at each of n positions (probability p/n each); finding it at position i costs i comparisons. If absent (probability 1 − p), n comparisons are used.

$$C_avg(n) = \Sigma\ (i = 1 \text{ to } n)\ i \cdot (p/n) + n \cdot (1 - p)$$
$$= (p/n) \cdot n(n+1)/2 + n(1 - p)$$
$$= p(n+1)/2 + n(1 - p)$$

Check: p = 1 gives (n+1)/2 (successful search); p = 0 gives n (always unsuccessful).

For n = 1,000,000 and p = 0.5: C_avg = 0.5 × 1,000,001 / 2 + 1,000,000 × 0.5 = 250,000.25 + 500,000 = **750,000.25 comparisons ≈ 0.75n.**

For comparison, S2 needs at most 20 comparisons for the same n, so binary search is about 37,500 times cheaper on average here (provided the index is kept sorted).

### (f) Prove an Asymptotic Bound

$f(n) = 3n^2 + 10n \log_2 n + 50$. Definition: $f(n) = \Theta(n^2)$ if there exist $c_1, c_2 > 0$ and $n_0$ such that $c_1 n^2 \le f(n) \le c_2 n^2$ for all $n \ge n_0$.

**Lower bound.** For all $n \ge 1$, $10n \log_2 n \ge 0$ and $50 > 0$, so $f(n) \ge 3n^2$. Hence $c_1 = 3$ works.

**Upper bound.** We need $10n \log_2 n + 50 \le n^2$ for large n. Take $n \ge 64$: $\log_2 n \ge 6$, and $\log_2 n \le n/10$ for $n \ge 64$ (at n = 64: $6 \le 6.4$, and n/10 grows faster than $\log_2 n$). Then $10n \log_2 n \le n^2$. Also $50 \le n^2$ for $n \ge 8$. Check n = 64: $10 \cdot 64 \cdot 6 + 50 = 3{,}890 \le 4{,}096$ ✓; since $n^2 - 10n \log n$ is increasing for $n \ge 64$, it continues to hold.

$$f(n) \le 3n^2 + n^2 = 4n^2 \quad \text{for all } n \ge 64$$

**Conclusion: $c_1 = 3$, $c_2 = 4$, $n_0 = 64$** give $3n^2 \le f(n) \le 4n^2$, so $f(n) = \Theta(n^2)$. (A simpler but looser choice: since $\log_2 n \le n$ and $1 \le n^2$, $f(n) \le 63n^2$ for $n \ge 1$, i.e. $c_2 = 63$, $n_0 = 1$.)

**f(n) is not O(n).** Suppose, for contradiction, that $f(n) \le c \cdot n$ for some constant $c > 0$ and all $n \ge n_0$. Since $f(n) \ge 3n^2$, we would get $3n^2 \le c \cdot n$, i.e. $n \le c/3$ for all $n \ge n_0$. This is false for any $n > \max(n_0, c/3)$. Contradiction. Therefore $f(n) \notin O(n)$ (in fact $f(n) = \Omega(n^2)$ and grows strictly faster than n).

### (g) Scalability Decision

n = 1,000,000, $\log_2 n \approx 19.93$, rate = $10^8$ operations/s, nightly window = 10 min = 600 s (= $6 \times 10^{10}$ operations).

| Module | Operation count | ≈ Time | Within 10 min? | Growth |
|---|---|---|---|---|
| M1 | $n(n-1)/2 = 4.99999 \times 10^{11}$ | 5,000 s ≈ 83.3 | NO – fails | $\Theta(n^2)$ |

| | | | | |
|---|---|---|---|---|
| | | min | | |
| M2 | $2^{20} - 1 = 1{,}048{,}575 \approx 1.05 \times 10^6$ | 0.0105 s | Yes | $\Theta(n)$ |
| S2 (per search) | 20 comparisons | ≈ 0.2 µs | Yes | $\Theta(\log n)$ |
| M4 current | $2n^2 - n \approx 2.0 \times 10^{12}$ | ≈ 20,000 s ≈ 5.6 h | NO – fails | $\Theta(n^2)$ |
| M4 redesign | $n \log n + n \approx 2.09 \times 10^7$ | 0.21 s | Yes | $\Theta(n \log n)$ |
| M5 | $\approx n \log n (\log n+1)/2 \approx 2.1 \times 10^8$ | ≈ 2.1 s | Yes | $\Theta(n \log^2 n)$ |

**Additional finding:** the nightly report generator $f(n) = 3n^2 + 10n \log n + 50$ needs about $3.0 \times 10^{12}$ operations ≈ 30,000 s ≈ 8.3 h at $n = 10^6$. It also fails the window, although it was not in the list of audited modules.

### Modules that fail the 10-minute window

M1 (≈ 83 min) and the current M4 design (≈ 5.6 h) fail. Both are quadratic, so a 10× increase in catalogue size (from 100,000 to 1,000,000) increases their cost 100×: M1 goes from about 50 s to about 5,000 s.

### Recommendations

- **M1 – redesign (mandatory).** Replace the all-pairs comparison with hashing: insert each product's key (title, seller) into a hash table and mark a product as duplicate if its key already exists. Expected time $\Theta(n)$ (≈ $10^6$ operations, ~0.01 s) with $\Theta(n)$ extra space. Alternative: sort by (seller, title) in $\Theta(n \log n)$ and compare adjacent entries, using little extra space.
- **M4 – adopt the proposed redesign (mandatory).** Removing replicated partitions changes $T(n) = 4T(n/2) + n = \Theta(n^2)$ to $T(n) = 2T(n/2) + n = \Theta(n \log n)$: about 20,000 s → 0.21 s, roughly a $10^5$× improvement. Fault tolerance previously supplied by replication should be provided by storage-level replication instead of by recomputing replicated data.
- **Report generator – redesign (recommended).** The $3n^2$ term must be removed (e.g., incremental aggregation, indexed/hashed grouping, or sort-based processing) to bring it to $O(n \log n)$.
- **M5 – acceptable, optional optimisation.** About 2 s at $n = 10^6$, but $\Theta(n \log^2 n)$ arises from re-sorting at every level. Sort once ($\Theta(n \log n)$) or use a merge-based approach in which partial results are already sorted, giving $T(n) = 2T(n/2) + n = \Theta(n \log n)$.
- **M2 and S2 – keep.** M2 is linear (the 'n log n' claim is wrong) and S2 is logarithmic. S1 (linear scan, ≈ 0.75n comparisons on average) should be replaced by S2 wherever a sorted index can be maintained.

□

### Appendix – Verification Program (Python) and Output

The program below implements the modules/recurrences and checks the derived closed forms. It was executed and the output is shown after it.

```
import math


def m1(n):
    c = 0
    for i in range(0, n - 1):
        for j in range(i + 1, n):
            c += 1
    return c


def m2(n):
    c, i = 0, 1
    while i < n:
        c += i
        i *= 2
    return c
```

```
def s2(n):            # T(n)=T(n/2)+1, T(1)=1
   return 1 if n <= 1 else s2(n // 2) + 1


def t4_current(n):    # 4T(n/2)+n
   return 1 if n <= 1 else 4 * t4_current(n // 2) + n


def t4_redesign(n):   # 2T(n/2)+n
   return 1 if n <= 1 else 2 * t4_redesign(n // 2) + n


def t5(n):            # 2T(n/2)+n log2 n
   return 1 if n <= 1 else 2 * t5(n // 2) + n * math.log2(n)


print("Brute-force check of M1:", all(m1(k) == k*(k-1)//2 for k in range(2, 200)))
print("M2(1024) =", m2(1024), "(n-1 =", 1023, ")  vs n log2 n =", 1024*10)
print("S2 closed form check:", all(s2(2**k) == k + 1 for k in range(0, 21)))
print("M4 current exact check (2n^2-n):", all(t4_current(2**k) == 2*4**k - 2**k for
k in range(0, 15)))
print("M4 redesign exact check (n log n + n):", all(t4_redesign(2**k) == k*2**k +
2**k for k in range(0, 15)))


RATE = 1e8
n = 1_000_000
L = math.log2(n)
rows = [
("M1 (n(n-1)/2)",         n*(n-1)//2),
("M2 (2^ceil(lg n)-1)",   2**math.ceil(L) - 1),
("S2 (floor(lg n)+1)",    math.floor(L) + 1),
("M4 current (2n^2-n)",   2*n*n - n),
("M4 redesign (n lg n+n)", n*L + n),
("M5 (n lg n (lg n+1)/2 + n)", n*L*(L+1)/2 + n),
("Report f(n)",           3*n*n + 10*n*L + 50),
]
print("\nn = 1,000,000, 1e8 ops/s, window = 600 s")
for name, ops in rows:
   t = ops / RATE
   print(f"{name:30s} ops = {ops:16,.0f}   time = {t:12.4f} s   {'FAILS' if t > 600
else 'OK'}")


p = 0.5
print("\nS1 average (n=1e6, p=0.5):", p*(n+1)/2 + (1-p)*n)
```

# Output:

```
Brute-force check of M1: True
M2(1024) = 1023 (n-1 = 1023 )  vs n log2 n = 10240
S2 closed form check: True
M4 current exact check (2n^2-n): True
M4 redesign exact check (n log n + n): True


n = 1,000,000, 1e8 ops/s, window = 600 s
M1 (n(n-1)/2)                 ops =  499,999,500,000   time =    4999.9950 s
FAILS
M2 (2^ceil(lg n)-1)           ops =        1,048,575   time =       0.0105 s   OK
S2 (floor(lg n)+1)            ops =               20   time =       0.0000 s   OK
M4 current (2n^2-n)           ops = 1,999,999,000,000   time =   19999.9900 s
FAILS
M4 redesign (n lg n+n)        ops =       20,931,569   time =       0.2093 s   OK
M5 (n lg n (lg n+1)/2 + n)    ops =      209,599,497   time =       2.0960 s   OK
Report f(n)                   ops = 3,000,199,315,736   time =   30001.9932 s
FAILS


S1 average (n=1e6, p=0.5): 750000.25
```